

ALAN’S CODING DARK FACTORY

Agentic Engineering Workflow

The complete operational playbook for building production systems with AI agents

Version 4.2 | April 2026
Derived from Project-DG (Canopy Stablecoin Analytics)

[L] Lopopolo	[C] Cherny	[W] Willison	[H] Hashimoto	[T] Taylor	[N] Carlini	[R] Schaad
--------------	------------	--------------	---------------	------------	-------------	------------

What changed in v4.2

This edition preserves the original 0→8 stage pattern, binary gates, and artifact chain while integrating seven guiding lenses: Lopopolo, Cherny, Willison, Hashimoto, Bret Taylor, Nicholas Carlini, and Raphael Schaad.

The primary improvement over v4.1 is Appendix E — the AGENTS.md file is now a production-grade, copy-pasteable operating contract with full structured sections, precise agent role boundaries, security protocols, inline checklists, and hero prompt starters. It is no longer a summary — it is the file you drop into a repository.

The Coding Dark Factory

What makes the Dark Factory interesting is that it treats software less like an artisanal craft and more like an industrial process. Most software teams still behave as if coding were scarce and judgment were abundant — embedding intelligence inside execution through sprints, standups, and endless in-flight correction. But once AI makes implementation cheap, that allocation looks backwards.

The scarce thing is no longer code. It is specification.

In a dark factory, you do not improvise on the line. You define the pattern, tolerances, tooling, and quality gates before the machines run. If the batch is wrong, you do not congratulate yourself on a clever midstream fix — you conclude the upstream system failed. The error was in the pattern, not the stitching.

That is why this workflow feels severe. The PRD defines intent. The reference guide defines implementation truth. The build plan defines execution order. The coding agent is not there to think through ambiguity heroically. It is there to execute a constrained system with high fidelity. The workflow is less a management process than an anti-chaos machine.

Who This Document Is For

Engineers, product builders, operators, and founders using AI coding agents to ship production systems. This is an end-to-end operational runbook for moving from objective definition to retrospective learning without losing control of correctness, security, user value, or design quality.

Use NotebookLM aggressively to deepen each hero lens when a section references one of the seven heroes. Each hero has a dedicated NotebookLM loaded with their interviews and talks. When you see a hero callout, treat it as a retrieval cue.

Part 1 — Core Principles & Workflow Map

The Seven Heroes at a Glance

Hero Lens	How the Workflow Uses It
[L] Ryan Lopopolo — The Harness Master Codex app creator	Constrain the environment so good behavior is mechanically easier than bad behavior. Shows up most strongly in determinism, binary gates, artifact discipline, reference-guide primacy, and the immutable harness.
[C] Boris Cherny — The Type-Driven Orchestrator Claude Code / CoWork creator	Plan before building. Break large work into explicit phases, tasks, and contracts. Shows up in stage design, decomposition, agent role boundaries, execution contracts, and Plan Mode.
[W] Simon Willison — The Agentic Architect Django creator	Assume failure, sandbox work, test empirically, and learn from what breaks. Shows up in adversarial review, sandboxing, validation, retrospectives, and real-environment verification.
[H] Mitchell Hashimoto — The Hammer Maker Terraform creator	Prefer practical primitives, zero-friction entry, and operator usefulness over elegance theater. Shows up in environment readiness, runbooks, scaffolding, and operational independence.
[T] Brett Taylor — The Product Machine Builder Google Maps creator	The company is a machine to produce happy customers. Shows up in objective setting, product-outcome mapping, design-partner discipline, and business-value validation.
[N] Nicholas Carlini — The Adversarial Reductionist Anthropic security researcher	Treat AI systems as attack surfaces under worst-case pressure. Shows up in prompt-injection thinking, trust boundaries, least privilege, exfiltration risk, and adversarial testing.
[R] Raphael Schaad — The Temporal Craftsman Cron founder	Design is behavior over time: latency, state changes, copy, hierarchy, and feel. Shows up in operator experience, UI states, runbook clarity, and workflow polish.

Core Operating Principles

Principle	What It Means in Practice
Truth layers are distinct	Source material, NotebookLM synthesis, the PRD, the approved reference guide, and the build plan serve different purposes. Do not collapse them. NotebookLM output is not canonical automatically.

Principle	What It Means in Practice
No code before the plan survives review	The adversarial review loop is not optional. A plan that has not survived independent review is not ready for implementation.
Approved reference guide defines implementation correctness	The PRD defines product intent. The approved reference guide defines implementation correctness. If code disagrees with the approved reference guide, retask implementation — do not change the guide.
Source quality determines plan quality	Only implementation-quality sources are acceptable. Reject marketing, stale tutorials, and conceptual fluff.
Every pivot gets written down immediately	Reality contradicting the plan is a learning event. Capture the delta before applying the fix.
Gates are binary	"Mostly passing" is failing. Unresolved items mean the gate did not pass.
AI agents have defined roles	NotebookLM ingests and synthesizes. Claude critiques and consolidates. Codex builds. Gemini attacks independently. Humans decide scope and risk.
Determinism is required	Explicit constants, versioned logic, fixed ranges, pinned dependencies, idempotent writes. No implicit defaults.
Implementation does not rewrite the plan	If building reveals a design change, that is a pivot. Stop, document, escalate.
Product outcome matters	A technically clean system that moves no customer or operator metric is still a workflow failure. [T]
Security is architectural	Do not rely on prompt wording or human vigilance as the primary defense. Separate data from instructions. [N]
Experience is operational behavior	Latency, copy, states, and runbook clarity are part of correctness — not afterthoughts. [R]

Workflow Map

Stage	Name	Primary Artifact
0	Define Objective	Objective Statement
1	Construct Reference System	Approved Coding Reference Guide
2	Generate Build Plan	Build Plan
3	Adversarial Review Loop	Risk Register
4	Execution Readiness	Readiness Report
5	Implementation	Working System + Learning Logs
6	Verification & Validation	Validation Report

Stage	Name	Primary Artifact
7	Stabilization & Runbook	Operational Runbook
8	Retrospective Learning Capture	Updated Workflow + Reference Guide

The Artifact Chain — Never Break It

Objective → Reference Guide → Build Plan → Risk Register → Readiness Report → Working System + Learning Logs → Validation Report → Runbook → Updated Workflow / Reference Guide.

A missing artifact is a gate failure for the stage that should have produced it. A stage does not end until its artifact exists and is complete. [L]

Part 2 — AI Agent Roles & Collaboration Protocol

Agent	Role — Do / Do Not
NotebookLM	DO: source ingestion, hero knowledge retrieval, structured Q&A, conflict discovery, consolidation input. DO NOT: adversarial review, direct implementation, automatic canonical authority.
Claude	DO: reference consolidation, plan review, adversarial attack, eval design, runbook drafting, retrospective synthesis. DO NOT: primary production code, schema authority, threshold decisions.
Codex	DO: plan-driven code, SQL, infrastructure, scaffolding, eval implementation. DO NOT: architecture decisions, threshold changes, correctness authority.
Gemini	DO: second-opinion adversarial review of the build plan — before seeing Claude output. DO NOT: any task after being shown Claude's framing. Contaminated independence = no independence.
Human / Workflow Owner	DO: scope calls, risk acceptance, pivot approval, production incident response. DO NOT: decisions already resolved in the approved reference guide, PRD, or build plan.

Rule: Never show Gemini Claude's output first

Independent second opinions are only useful when they are actually independent. Gemini must see the build plan before it sees Claude's risk framing. Otherwise the review is contaminated and loses discovery power.

Practical sequence: (1) Claude reviews build plan + reference guide. (2) Gemini reviews build plan only. (3) Merge findings. Any other order invalidates the independence. [W] [N]

Part 3 — Stage Runbooks

0

Define Objective

Lock the success definition before any agent, tool, or document is created.

Hero callout — Taylor + Cherny [T] [C]

Taylor sharpens the outcome question: whose problem gets better, and how will you know?

Cherny sharpens precision: define the contract before any implementation begins.

Required Outputs

- Success criteria — falsifiable, specific, measurable
- Constraints — cost ceiling, technology, compliance, timeline
- Scope boundary and explicit non-goals
- Target user/operator and economic beneficiary
- Measurable outcome hypothesis — which metric moves if this works?
- Trust boundary sketch — sensitive data, privileged actions, untrusted inputs

 **GATE 0**

Is success unambiguous technically and economically?

 PASS Any engineer reading the objective knows exactly when the project is done. Constraints and non-goals are explicit. Trust boundary is sketched.	 FAIL Stop. Refine until the success condition is unambiguous and the scope boundary is clear. Do not create any agent, tool, or document before Gate 0 passes.
--	---

1

Construct Reference System

Build the approved source of truth for implementation correctness.

Hero callout — Lopopolo + Taylor + Carlini + Schaad [L] [T] [N] [R]

Lopopolo demands explicitness — every rule checkable. Taylor asks whether the workflow serves a real user job. Carlini forces mapping of trusted versus untrusted inputs. Schaad defines states, messages, and latency expectations for any human-facing surface.

Reference Truth Stack

Source Corpus → NotebookLM synthesis → Draft Reference Guide → Approved Reference Guide (canonical). NotebookLM output is not canonical by default. It becomes authoritative only after consolidation, conflict resolution, and explicit approval.

Required Reference Guide Sections

Section	What It Must Define
Schemas	Every table, column, type. No TBD entries.
Constants	Token addresses, topic hashes, decimal precision, confirmation buffers — all verified.
Calculation rules	Every derived field formula. Explicit, no inference.
Thresholds	All numeric: match rate, supply delta, bytes billed ceiling, row-count tolerance.
Edge cases	Defined behavior for every failure scenario — no undefined behavior allowed.
Retry rules	Max attempts, backoff strategy, terminal vs retryable classification.
Idempotency rules	MERGE keys, deduplication logic, rerun behavior.
Failure handling	Run failure states, alert triggers, rollback policy.
Infrastructure config	Project, dataset, region, service account, IAM roles, scheduler config.
Trust boundaries [N]	Privileged actions, trusted vs untrusted input surfaces, exfiltration paths.
Product outcome metrics [T]	Which metric moves if this works? How will you measure it?
UX state expectations [R]	Operator-facing states, latency SLAs, error message copy for critical paths.



GATE 1

Can a coding agent implement without guessing?

 PASS Every schema, constant, threshold, calculation, edge case, retry rule, failure behavior, trust boundary, and UX state is explicitly defined. No material decision is left to agent judgment.	 FAIL Identify what is undefined. Return to iterative questioning for each gap. Do not proceed to Stage 2 with any ambiguous requirement.
---	--

2

Generate Build Plan

Convert the reference system into a deterministic, phase-by-phase execution contract.

Hero callout — Cherny + Lopopolo + Hashimoto [C] [L] [H]

Cherny contributes Plan Mode decomposition. Lopopolo contributes determinism and exactness. Hashimoto asks whether the plan reaches a usable, operable system with minimal friction instead of elegant dead weight.

Plan Mode Procedure [C]

Required for medium and hard tasks. Moves success rates from ~25% to ~75%.

1. Codex produces Options 1, 2, and 3 for each unresolved architectural decision.
2. Human reviews, selects, and documents rationale.
3. Codex produces a markdown to-do list based on the selection.
4. Only then: full build plan drafting begins.

Phase Design Rules

- Sequential execution — phases run in order. No parallel phases unless explicitly defined.
- Independent validation — each phase has its own validation checkpoint before the next begins.
- Minimal scope — each phase implements one capability. Never combine two.
- Reversible state — idempotent operations throughout. Every phase allows rollback.
- Black-box testability — phase correctness defined by inputs and outputs, not internal implementation. [H]

GATE 2

Can a coding agent execute this plan without making design decisions?

 PASS All phases defined. All tasks have exact commands and expected outputs. All dependencies enumerated. All exit gates are measurable. Phase correctness is black-box verifiable.	 FAIL Identify ambiguous phases or missing dependencies. Return to the reference guide if a design decision is unresolved. Re-run Plan Mode for unresolved architectural choices.
---	--

3

Adversarial Review Loop

Break the plan before execution. Risks found here cost minutes. The same risks at Stage 5 cost days.

Hero callout — Willison + Carlini [W] [N]

Willison says assume the plan is wrong and test in reality. Carlini says strip away the magic, model the attack surface, and evaluate worst-case adversaries instead of polite ones.

Assume the plan is wrong until proven otherwise. Every review cycle starts from the assumption that the plan has failures. The reviewer's job is to find them — not confirm the plan is good.

Review Cycle Procedure

5. Independent Claude review: full build plan + reference guide. Do not share with Gemini yet.
6. Independent Gemini review: build plan only — no reference guide, no Claude output.
7. Merge risk logs. Classify: Critical / Medium / Minor.
8. Resolve all Critical items. Document explicit acceptance rationale for deferred risks.
9. Update plan. Re-review any materially changed sections.
10. Repeat until zero unresolved critical risks remain.

Required Review Categories

Category	Review Question
Technical correctness	Does the plan contain incorrect logic?
Completeness	Are required steps missing?
Dependency risk	Are dependencies unavailable or unverified?
Execution / data / infrastructure risk	Will the system fail, corrupt data, or lose access at runtime?
Observability risk	Can failures go undetected?
Determinism / cost risk	Can repeated runs produce different results? Can a job exceed billing ceilings?

Category	Review Question
Prompt injection risk [N]	Does any untrusted input surface reach an agent with data access?
Exfiltration risk [N]	Is the Lethal Trifecta present? Is the exfiltration leg severed architecturally?
Privilege risk [N]	Can the agent take actions beyond what it needs for this task?
Product-outcome mismatch [T]	Does the plan actually produce the outcome the objective specifies?
Operator confusion [R]	What does the operator see under degraded or failure states?

 GATE 3	
Are there zero unresolved critical risks?	
 PASS Every risk is resolved, explicitly accepted with documented rationale, or formally deferred with a trigger condition.	 FAIL Continue the review loop. Return to Step 1 for any section materially changed after a prior review cycle.

4

Execution Readiness

The last gate before implementation. The environment must be ready before the first line runs.

Hero callout — Hashimoto + Lopopolo + Carlini + Schaad [H] [L] [N] [R]

Hashimoto emphasizes one-command operability and practical readiness. Lopopolo demands deterministic setup. Carlini demands least privilege and outbound-exfiltration review. Schaad reminds you that first-run failure messages and latency handling are part of readiness.

Sandbox containment verified — isolated environment confirmed before enabling autonomous execution [W]	☐ Pass/Fail
Lethal Trifecta reviewed — exfiltration leg addressed architecturally, not via prompt wording [N]	☐ Pass/Fail
Compute, storage, network, region, and quotas provisioned	☐ Pass/Fail

Service account created, minimum IAM granted, no secrets in repo, auth tested locally	☐ Pass/Fail
All external APIs and RPC endpoints reachable from the runtime environment	☐ Pass/Fail
Runtime version pinned, all libraries installed in .venv	☐ Pass/Fail
Repository structure complete, .env.example covers all variables, local .env populated	☐ Pass/Fail
make doctor passes — environment self-diagnosis confirms all configuration	☐ Pass/Fail
Validation and observability logic operational — row count, duplicate detection, freshness, alerts	☐ Pass/Fail
Product measurement instrumentation live — outcome metric is being captured	☐ Pass/Fail
UX failure messaging verified — degraded-state messages are clear and actionable [R]	☐ Pass/Fail
Dry run completes against bounded window without environment errors	☐ Pass/Fail
Determinism verified: same command twice → identical output	☐ Pass/Fail

 GATE 4

Can the system start without manual fixes, inside a verified sandbox?

 PASS

Sandbox confirmed. make doctor passes. All dependencies reachable. Dry run clean. Determinism verified. Trifecta addressed.

 FAIL

Fix the gap. Do not proceed to Stage 5 with any unresolved environment issue or unconfirmed sandbox.

5

Implementation

Execute the approved build plan exactly as specified. One phase at a time. No improvisation.

Hero callout — Lopopolo + Cherny + Willison [L] [C] [W]

Lopopolo forbids silent drift from the reference guide. Cherny keeps implementation bounded to the current phase. Willison requires empirical logging, manual simulation, and explicit learning capture when reality bites.

Execute the plan exactly. Do not improvise. If Codex encounters something that requires a plan change, that is a PIVOT — not a silent deviation. Stop. Document it. Escalate. [L]

Error Handling — Root Cause Automation [L]

- 11. Stop. Do not proceed to the next task.
- 12. Classify: code bug / environment issue / source accessibility / spec conflict.
- 13. Check the learning log — known fix? Apply it first.
- 14. Write one sentence: what failed and why. Before fixing.
- 15. Apply minimum change. Retry. Log root cause if resolved.
- 16. Ask: what harness rule, lint, or gate criterion makes this failure class physically harder to repeat? Document the answer as a Stage 8 input.
- 17. Spec change required? → PIVOT. Document and escalate before Codex continues.

 GATE 5

Does the system run without runtime errors?

 PASS

All phases executed. All tasks validated. All logs generated. All validation checkpoints pass. No silent deviations from the reference guide.

 FAIL

Apply the error handling procedure. Update the learning log. If a spec change is needed, that is a Pivot — escalate before continuing.

6

Verification & Validation

Confirm the system is correct, useful, secure, and trustworthy — not merely running.

Hero callout — Willison + Taylor + Carlini + Schaad [W] [T] [N] [R]

Willison contributes evidence over vibes. Taylor contributes business/operator outcome checks. Carlini contributes boundary and attack validation. Schaad contributes clarity of states, latency, and feedback under stress.

Domain	Tests	Threshold
Data accuracy	Reconciliation (Path A vs Path B), supply validation, duplicate detection, freshness, schema match, negative value detection, partition integrity	Match ≥99%, delta ≤0.01%, zero dupes, ≤8h stale, 100% schema, zero negatives
Reliability	Run twice → identical output	Zero drift between runs
Recovery	Interrupt mid-run → restart → verify no duplicate writes	Clean resume, MERGE prevents duplication

Domain	Tests	Threshold
Observability	Trigger each alert condition intentionally	All alerts fire within expected window
Retry	Simulate failures → verify backoff → verify stop	Max 3 retries, exponential backoff, clean stop
Security [N]	Permission boundary test, prompt injection test, exfiltration path test	All three clean
Product outcome [T]	Spot-check target metric against expected direction	Metric moves as hypothesized
Operator UX [R]	Degraded-state message clarity, latency under load, error feedback actionability	Messages clear, latency within SLA, errors actionable

 **GATE 6**

Do all validation thresholds pass?

 PASS Every eval passes. Data accuracy, reliability, recovery, observability, security, outcome, and UX checks all meet defined thresholds.	 FAIL Investigate and fix before promoting any run. Partial pass is a fail. Do not proceed to Stage 7 with any unresolved validation failure.
---	---

7

Stabilization & Runbook
Make the system independently operational. If it requires the builder every cycle, it is not done.

Hero callout — Hashimoto + Schaad + Taylor [H] [R] [T]
Hashimoto wants a practical operating machine. Schaad wants the operator experience to feel clear under both healthy and unhealthy states. Taylor wants the machine to keep producing value, not just uptime.

Burn-in Rule
Run N consecutive scheduled cycles without a failure. Any failure resets the streak to zero. Partial streaks do not count. A partial streak proves the system was stable once and then broke — not that it is stable.

Runbook Required Sections

Section	Must Contain
Startup steps	Exact commands. One-command entry point first.
Daily monitoring	What to check, what "healthy" looks like, which queries to run.
Alert response	3-step diagnostic + resolution path per alert type. Exact commands.
Known failure modes	Root cause + fix per failure type. Drawn from learning log.
Escalation path	When to escalate vs debug. What to include when escalating.
Stale-run handling	How to identify, safely clear, and verify ready.
Security incidents [N]	Response steps, isolation procedure, who to notify.
Operator confusion cases [R]	Degraded-mode playbooks. What to say, what to do.
Outcome monitoring [T]	Which metric, cadence, threshold, and owner.
Shutdown, version control, backup/restore	Safe stop, version history, restore procedure.

 GATE 7	
Does the system run unattended for a full burn-in cycle?	
 PASS N consecutive successful scheduled runs. All alerts tested and confirmed to fire. Runbook written, reviewed, and verified against actual system behavior.	 FAIL Investigate the failure that broke the burn-in streak. Fix, reset the counter to zero, and restart.

8

Retrospective Learning Capture

Convert build experience into reusable capability. This is where the workflow improves.

Hero callout — Willison + Lopopolo + Taylor [W] [L] [T]

Willison makes failure data sacred. Lopopolo turns repeated errors into workflow rules. Taylor asks whether the work actually moved outcomes. Good incidents become future constraints.

Knowledge Capture Template

Incident Description: [one sentence]

Root Cause: [underlying cause, not symptom]

Resolution: [exactly what changed]

Prevention: [rule or check added to ref guide, workflow, or runbook]

Harness Update [L]: [what lint, gate criterion, or constraint makes this failure class physically harder to repeat?]

GATE 8	
Is the workflow improved and can the next build avoid the same failures?	
PASS Reference guide, workflow, and runbook all reflect lessons from this build. Harness updates encoded. Retrospective report published. Version incremented.	FAIL Capture the missing lessons. The workflow is not complete until Stage 8 is complete. Projects that skip this stage transfer rediscoverable failures to the next build.

Appendix A — The Level 5 Harness

This appendix summarizes the repository-level operating contract that sits underneath the workflow. The workflow tells the team what to do stage by stage. The harness tells humans and agents how they are permitted to behave while doing it. When they conflict, the stricter rule wins.

Harness Rule	Meaning
Human-write-only constraints [L]	Agents may not modify harness files, lint rules, or permanent constraints to get a PR to pass. If the harness blocks you: surface the conflict. Do not route around it.
0% human implementation [L]	Humans design constraints and intent. Agents generate and regenerate implementation. A human touching implementation code is a process failure.
Plan Mode before implementation [C]	Medium and hard tasks require options, tradeoffs, a selected plan, and a markdown to-do list before code is touched.
Red / Green TDD [L] [W]	Write a failing test before implementation. No valid failing test = no merge. This is not optional.
Sandbox first [W]	Autonomous execution is allowed only inside contained environments. Containment first. Velocity second.
Root-cause automation [L]	Do not patch isolated bugs in a loop. Add a lint, test, rule, or constraint that makes the failure class physically harder to repeat.
Repo-centric truth [L]	If it is not in the repo, it does not exist to the agent. Slack, chat, and memory are invisible.
Sub-minute inner loop [L]	Build and test loops must complete in under 60 seconds. If exceeded: stop feature work, fix the build graph. Stalled agents waste context windows.
Security by architecture [N]	Prompt wording does not count as a primary security mechanism. Architecture-level controls only.
Scaffolding is temporary [C]	Label any guardrail TEMPORARY. State the removal trigger. Delete it when model capability makes it unnecessary.

Operational Mantras

Constrain hard. Plan clearly. Test honestly. Attack your own system.

Measure customer or operator value — not feature count.

Separate data from instructions.

Never let a silent deviation rewrite correctness.

Latency, copy, and feedback are part of correctness.

Every retrospective must tighten either the harness or the workflow.

The scarce resource is not code. It is specification.

Appendix B — Quick Reference

Pre-Execution Checklist

Objective is unambiguous technically and economically	☐ Pass/Fail
Reference guide covers schemas, constants, thresholds, edge cases, retry rules, failure handling	☐ Pass/Fail
Trust boundaries, privileged actions, and untrusted input surfaces are defined	☐ Pass/Fail
Product outcome metric and UX state expectations are defined	☐ Pass/Fail
Build plan is deterministic with measurable exit gates per phase	☐ Pass/Fail
Plan Mode completed for all medium/hard architectural decisions	☐ Pass/Fail
Claude adversarial review complete — risk log produced independently	☐ Pass/Fail
Gemini independent review complete — before seeing Claude output	☐ Pass/Fail
All critical risks resolved or explicitly accepted with rationale	☐ Pass/Fail
Sandbox containment verified before enabling autonomous execution	☐ Pass/Fail
Lethal Trifecta reviewed — exfiltration leg addressed architecturally	☐ Pass/Fail
Environment configured and make doctor passes	☐ Pass/Fail
External dependencies reachable from runtime environment	☐ Pass/Fail
Dry run completes without environment errors	☐ Pass/Fail
Determinism verified: same command twice → identical output	☐ Pass/Fail

Determinism Checklist

All block ranges are fixed — not derived from live chain head	☐ Pass/Fail
All token addresses come from versioned config — not inline constants	☐ Pass/Fail
All production writes use MERGE on canonical keys — no plain INSERT	☐ Pass/Fail
maximum_bytes_billed enforced on every BigQuery job	☐ Pass/Fail
decode_version recorded on every row	☐ Pass/Fail
run_id links every operational record to the originating run	☐ Pass/Fail
Running the same command twice produces identical output	☐ Pass/Fail
No relative timestamps or implicit defaults in production queries	☐ Pass/Fail
All dependencies pinned to explicit versions	☐ Pass/Fail

Sensitive actions restricted by least-privilege boundaries	☐ Pass/Fail
--	------------------------------------

Appendix C — NotebookLM Prompt Starters by Hero

When a decision is hard, load the relevant NotebookLM and use the prompt starter as your entry point. These are retrieval cues, not full prompts — expand them with the specific context of your current stage.

Hero	NotebookLM Prompt Starter
[L] Ryan Lopopolo — The Harness Master Codex app creator	"What mechanical constraint or lint would make this failure class physically harder to repeat?"
[C] Boris Cherny — The Type-Driven Orchestrator Claude Code / CoWork creator	"What are the 3 cleanest implementation options, and what markdown to-do list would align the team before coding?"
[W] Simon Willison — The Agentic Architect Django creator	"How would this fail in reality, and what empirical test would reveal that fastest?"
[H] Mitchell Hashimoto — The Hammer Maker Terraform creator	"What is the most practical primitive or one-command setup that lowers friction without hiding important complexity?"
[T] Brett Taylor — The Product Machine Builder Google Maps creator	"What specific customer or operator outcome improves if we ship this? What is the strongest validation signal?"
[N] Nicholas Carlini — The Adversarial Reductionist Anthropic security researcher	"If I treat this as an attack surface instead of a product, what is the simplest exploit path across trust boundaries?"
[R] Raphael Schaad — The Temporal Craftsman Cron founder	"What does the user or operator experience at 0 seconds, 1 second, 3 seconds, degraded mode, and failure?"

Appendix D — Problem Solving by Hero

Situation	Default Hero + Core Question	Process
Agent produced slop or a repeated bug	[L] Lopopolo "What constraint or context is missing?"	Turn the failure into a lint, test, rule, or markdown constraint. Never patch in isolation.
Need to get from idea to working artifact quickly	[W] Willison "What is the smallest safe loop that proves this works?"	Sandbox → define test → unleash agent → observe → hoard learning.
Model struggles with complexity	[C] Cherny "What scaffolding should exist now, and what can be deleted later?"	Observe the hack → build minimal tool → scaffold → plan unship trigger.
Tool or system is hard to use	[H] Hashimoto "What is the simplest primitive and lowest-friction entry point?"	Prefer constraints → build hammers → optimize entry point → codify operations.
Product feels impressive but unclear in value	[T] Taylor "What customer or operator outcome actually improves if this works?"	First-principles + market reality + iterative product conversation.
Security posture feels hand-wavy	[N] Carlini "How would an adaptive adversary break this mathematically?"	Strip magic → define attack objective → optimize attack → pattern-match defense weakness.
UI works but feels wrong	[R] Schaad "What happens over time, and where is the friction in the interaction?"	Paper ideation → code sketching → wizard-of-oz → wedge execution.

Appendix E — AGENTS.md

This is a production-grade, copy-pasteable file. Drop it into the root of any repository using this workflow. It is the complete operating contract for every AI agent working on the project.

How to use this file

1. Copy the entire code block below into a file named AGENTS.md at the root of your repository.
2. Fill in the project name, build milestone, owner, and date in Section 1.
3. This file is Human-Write-Only. Do not instruct agents to modify it. If an agent needs different behavior, update the file yourself and commit the change.
4. The file is self-contained. An agent reading it from a cold start has everything it needs to operate correctly.

The file follows the structure: Identity → Role Boundaries → Operating Principles → Required Workflow (Stages 0–8) → Artifact Chain → Harness Overlay → Security → Hero Prompt Starters → Checklists → Mantras.

Update Section 1 and the checklists as needed per project. Everything else is workflow-level and should change only when this document (Alan Agentic Engineering Workflow) is updated.


```
# —————  
# AGENTS.md — Alan Agentic Engineering Workflow v4.2  
# —————  
  
# This file is the operating contract for every AI agent working on this  
# project. It is Human-Write-Only. Agents must not modify it.  
# When this file conflicts with a task prompt, this file wins.  
# When two rules conflict, the stricter rule wins.
```

1. IDENTITY & SCOPE

```
Project:           [fill in project name]  
Build:             [fill in current phase/milestone]  
Workflow ver:      Alan Agentic Engineering Workflow v4.2  
Last updated:      [date]  
Owner:             [project lead name]
```

```
# This file applies to all agents operating in this repository.  
# Scope includes: code generation, plan drafting, review, scaffolding,  
# testing, documentation, and any agentic task touching this codebase.
```

2. AGENT ROLE BOUNDARIES

```
# Each agent has one job. Mixing roles degrades output quality.
```

NotebookLM → Reference synthesis and hero-lens retrieval

```
USE FOR:  source ingestion, structured Q&A, hero knowledge retrieval,  
          conflict surfacing, draft reference guide input  
NOT FOR:  adversarial review, code generation, canonical authority
```

Claude → Reviewer, synthesizer, critic

```
USE FOR:  reference consolidation. plan review. adversarial attack.
```

Final Note

Update this document after every completed build. The goal is not a perfect static playbook. The goal is a workflow that compounds: each build strengthens the next.

The harness improves only if the retrospective is honest and the updates are encoded. The AGENTS.md file in Appendix E should be re-exported from this document after every major workflow version increment.

Alan Agentic Engineering Workflow v4.2 | April 2026